

kernel mode

- ↳ direct access to all hardware resources
- ↳ OS runs in kernel mode!

user mode

- ↳ limited access to hardware resources
- ↳ applications / software

kernel

- ↳ deals with hardware issues
- ↳ provides system call interface
- ↳ special code for interrupt handlers, device drivers

monolithic

- ↳ one big program as kernel
- ↳ advantages
 - ↳ good performance
 - ↳ well understood

- ↳ disadvantages
 - ↳ highly complex components
 - ↳ complicated internal structure

microkernel

- ↳ small and clean
- ↳ only provides basic/essential facilities
 - ↳ ipc
 - ↳ threads
 - ↳ address space
- ↳ high level OS services
 - ↳ built on top of kernel
 - ↳ uses ipc to communicate

advantages

- ↳ more robust and extensible
- ↳ better isolation and protection between kernel and high level services
- ↳ OS advantages
 - ↳ better performance

memory

- ↳ text → code
- ↳ data → global variables
- ↳ stack
 - ↳ return address
 - ↳ arguments for function
 - ↳ storage for local variables
 - ↳ old stack pointer / frame pointer
 - ↳ saved registers
 - ↳ heap
 - ↳ dynamic memory allocated mem
 - ↳ malloc
 - ↳ allocated at runtime, size not known
 - ↳ no definite deallocation time

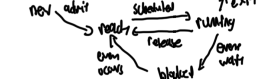
process abstraction

- ↳ dynamic abstraction for executing program.
- ↳ memory / hardware / OS context

process ID

- ↳ used to identify processes

5 state process model



- ↳ CPU → 5 processes in running state

- ↳ one transition at a time!

- ↳ many → 5 in processes running

- ↳ possible process transitions

POS and PWS tables

- ↳ stores process information

- ↳ kernel stores a table for all PWS

- ↳ info can be called process table entry

- ↳ can sleep in kernel

- ↳ switches from user mode to kernel mode

sleep

- ↳ user program knows call

- ↳ library call places special number in a designated location

- ↳ library call executes special instruction

- ↳ switch to kernel

- ↳ saves CPU state

- ↳ determine appropriate execution unit

- ↳ set on number

- ↳ dispatcher

- ↳ special handler is executed

- ↳ after execution

- ↳ restore CPU state and return to library call

- ↳ switch back to user mode

- ↳ library call returns back to user program

exception

- ↳ divide by 0 / illegal memory access / null address

- ↳ OS advantages
 - ↳ better performance

- ↳ OS advantages
 - ↳ better performance

- ↳ OS advantages
 - ↳ better performance

- ↳ OS advantages
 - ↳ better performance

- ↳ OS advantages
 - ↳ better performance

- ↳ OS advantages
 - ↳ better performance

- ↳ OS advantages
 - ↳ better performance

- ↳ OS advantages
 - ↳ better performance

- ↳ OS advantages
 - ↳ better performance

- ↳ OS advantages
 - ↳ better performance

- ↳ OS advantages
 - ↳ better performance

- ↳ OS advantages
 - ↳ better performance

- ↳ OS advantages
 - ↳ better performance

- ↳ OS advantages
 - ↳ better performance

- ↳ OS advantages
 - ↳ better performance

forking

- ↳ creates a new process

- ↳ returns PID of child

- ↳ returns 0 for child

- ↳ child is copy of parent

- ↳ memory is not shared

- ↳ unless shm is used

- ↳ both parent and child continue execution line after fork

process

- ↳ unique address space of child

- ↳ create kernel process structures

- ↳ exp. context in process table

- ↳ copy kernel env of parent

- ↳ init child process context

- ↳ copy memory regions of parent

- ↳ program, data, stack

- ↳ optimized via copy on write

- ↳ allocated shared resources

- ↳ init hardware context for child

- ↳ copy registers from parent etc.

- ↳ can run child after wards!

exec

- ↳ replaces current process!

- ↳ no new process is created

continuing both:

- ↳ spawn child processes

- ↳ parent stays to accept customer requests

init process

- ↳ PID = 1

- ↳ watches for other processes and responds when needed

- ↳ fork creates process tree

exit

- ↳ good

- ↳ not good (error exit)

- ↳ status returned to parent

wait (int x bytes)

- ↳ stores return status

- ↳ return PID of terminated child

zombies

- ↳ appears after process exits

- ↳ can only be cleaned after parent calls wait

- ↳ cannot be killed → already dead

- ↳ can fill up process table

orphan

- ↳ parent process dies but child

- ↳ when child dies → signal sent to init which does wait to clean

concurrent

- ↳ virtual parallelism

- ↳ single core, single CPU

- ↳ parallelism

- ↳ multiple CPU / multi-core CPU

- ↳ switching between processes requiring context switching

- ↳ some overhead incurred while switching

- ↳ scheduler

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ process activity

- ↳ CPU activity

- ↳ I/O

- ↳ requesting/receiving service from IO device

- ↳ init / read file / write etc.

- ↳ process environment

- ↳ batch processing

- ↳ no user interaction → no need to be responsive

- ↳ interactive

- ↳ user interacts with system

- ↳ should be responsive

- ↳ low and consistent response time

- ↳ real-time processing

- ↳ have deadline to meet

- ↳ usually periodic process

- ↳ important criteria for scheduling!

- ↳ priority scheduling

- ↳ should optimize system resource properly

offload

- ↳ turnaround time → finish arrival

- ↳ throughput → ready wait time

- ↳ CPU utilization

- ↳ response time → run arrived

- ↳ predictability variation in response time

scheduling process

- ↳ scheduler is triggered

- ↳ if context switch is needed

- ↳ time context of current process

- ↳ then process to blocked / ready queue

- ↳ pick process based on algo

- ↳ set up context of P

- ↳ run P

batch processing algo

- ↳ FCFS

- ↳ process run until finished / blocked

- ↳ process removed and placed to back of queue

- ↳ process X will eventually get to run

- ↳ no starvation

- ↳ SJF

- ↳ select task with smallest total CPU time

- ↳ if CPU time not equal, take

- ↳ need to queue

- ↳ minimizes waiting time

- ↳ can cause large jobs to never run through

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

priority inversion

- ↳ lower priority task preempt higher priority task

- ↳ MFLB

- ↳ A & B → A runs

- ↳ A = B → A and B runs in parallel

- ↳ new jobs higher priority

- ↳ used up execution → priority

- ↳ wait / gives up → priority

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

pipelining

- ↳ pipe from output of one process to input of another process

- ↳ only

- ↳ one to write to

- ↳ one to read from

- ↳ like an assembly line

- ↳ must queue data in order

- ↳ pipe full → writer wait

- ↳ pipe empty → reader wait

- ↳ Round Robin

- ↳ makes scheduling decision

- ↳ uses scheduling algo to make the decision

- ↳ Round Robin

Extra notes:

priority inversion:

one high and one low using the same resource. The other task should be in the medium.

Interweaving

↳ for minimum value

↳ 1st does 1st w

↳ 2nd runs overwriting except last loop
every-time else finishes everything

1st does up and stores

↳ 2nd loads, getting ready at 1st of again

↳ 1st finishes very early

↳ 2nd does it and store.

Dining Philosopher

↳ releasing the forks of each prevents deadlock

Busy waiting

↳ repeatedly checking a condition

↳ consumes CPU!!

read question!!

↳ single core + 1000 threads at a time!

↳ regardless of how many threads the
process/kernel has

mutual Exclusion

↳ $S=1$

wait(S)

// S

signal(S)

k on current process

S++

wait(S)

// S

signal(S)